



AmigaED 4.0 Quick Manual

Quick Reference & Getting Started Guide

AmigaED is a cross-platform C/C++ and m68k Assembler IDE for classic Amiga development.

Table of Contents

1. Introduction
2. Two Ways to Build Your Code
3. Toolbar Reference
4. Mode 1: Ad-hoc One File Compilation
5. Mode 2: Project Driven Compilation
6. Quick Start: From Template to Clean Build
7. The Preferences Editor
8. The Editor's Context Menu
9. The Compiler Output Pane
10. Suggested Compiler & Linker Switches
11. Recommendations for Amiga C Programmers

Introduction



AmigaED

Two Ways to Compile

A short, illustrated guide to Ad-hoc One File Compilation, Project Driven Compilation, and the Preferences Editor.

AmigaED is a cross-platform C/C++ and m68k Assembler IDE for classic Amiga development.

Two Ways to Build Your Code

AmigaED supports two independent ways to turn your source code into an AmigaOS-runable program. Both are always available side by side, so use whichever fits what you are doing right now.

1. Ad-hoc One File Compilation – open a single source file and compile it directly, with no project setup at all. Best for a quick test, a one-off tool, or trying out an idea.

2. Project Driven Compilation – group multiple files (sources, headers, assembler, icons, and so on) into a Project. AmigaED tracks them, generates Makefiles for you, and builds the whole thing with one click. Best for anything with more than one file, a GUI (ReAction/MUI), or a program you will come back to later.

	Ad-hoc	Project
Best for	Quick single-file tests	Multi-file / GUI programs
Setup needed	None, just open a file	Create or import a project
Multiple files linked together	No	Yes
Makefiles	Not generated	Auto-generated (gcc, vbcc, SAS/C)
Persists between sessions	No	Yes (.aep project file)

Toolbar Reference

Every toolbar button mirrors a menu entry exactly - the toolbar is purely a shortcut for the entries you reach for most often, never a separate feature of its own.



#	Button	Menu equivalent	What it does
1	New	File → New	Opens a fresh, empty editor tab.
2	Open	File → Open...	Opens an existing file in a new tab.
3	Save	File → Save	Saves the current tab.
4	Save As	File → Save As...	Saves the current tab under a new name/location.

5	Print	File → Print file...	Prints the current file.
6	Undo	Edit → Undo	Undoes the last edit.
7	Redo	Edit → Redo	Redoes the last undone edit.
8	Cut	Edit → Cut	Cuts the current selection.
9	Copy	Edit → Copy	Copies the current selection.
10	Paste	Edit → Paste	Pastes the clipboard contents.
11	Search	(context menu) Search and Replace...	Opens the Find/Replace panel for the current tab.
12	Goto Line	Navigation → Goto Line...	Jumps straight to a given line number.
13	Matching bracket	Navigation → Goto matching bracket	Jumps to the bracket/brace/parenthesis matching the one under the caret.
14	Compile	Build → Compile...	Ad-hoc single-file compile (Mode 1) - see below.
15	Build Project	Build → Build Project	Project build (Mode 2) - see below.
16	Clean Project	Build → Clean Project	Removes the current project's build artifacts (object files, executable, icon).
17	Open Shell	Build → Open Shell	Opens the system's default command line, starting in the current project's folder (or the configured "Projects root" if none is loaded).
18	Start Emulator	Tools → Emulator → Start default Workbench in UAE...	Launches your configured UAE emulator.
19	Stop Emulator	Tools → Stop running Emulation...	Stops the running emulator instance.
20	Exit	File → Exit	Closes AmigaED.

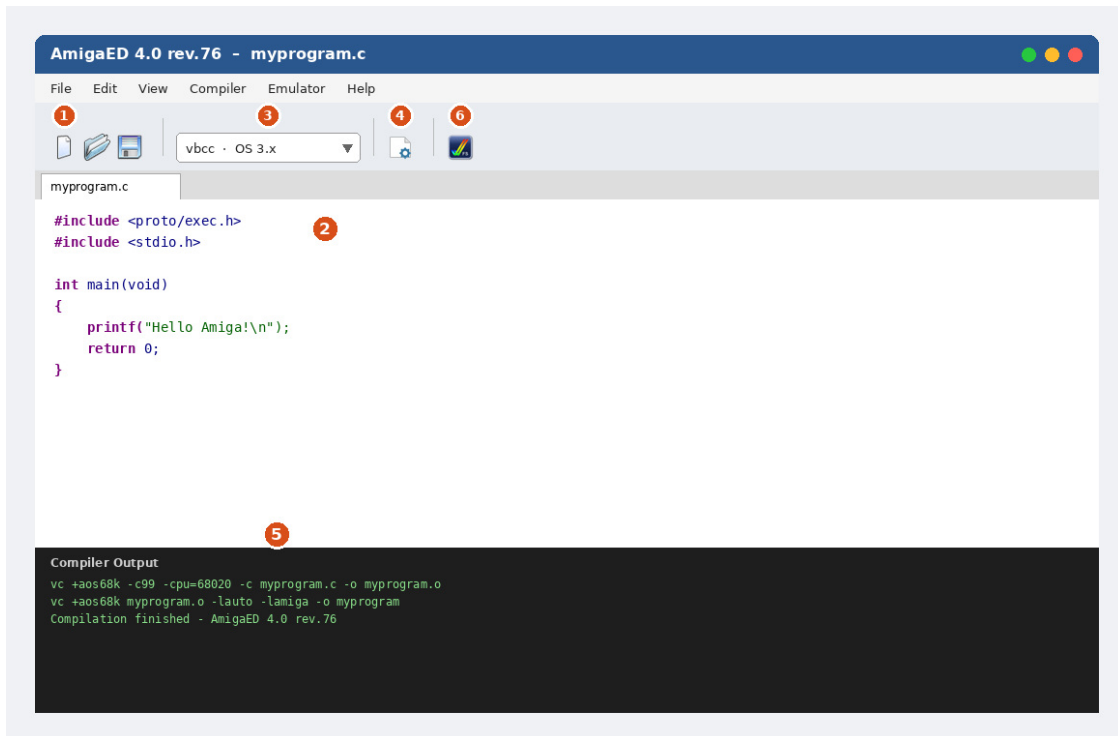
Warning: Open Shell gives you an unrestricted command line in the project's own folder - including the Makefiles and every file a build reads or produces. Editing, renaming, or deleting files there by hand, outside of AmigaED's own actions, can leave the project in a state that no longer matches what AmigaED expects, and may cause the next Build Project or Clean Project to fail or behave unexpectedly. Treat the shell as a power-user escape hatch, not part of the normal edit/build cycle.

Note: Start/Stop Emulator automatically grey out and enable each other depending on whether an emulator instance is currently running - including one AmigaED itself didn't start (e.g. left open on purpose from a previous session, or launched outside AmigaED entirely).

Mode 1: Ad-hoc One File Compilation

Use this when you just want to write and test a single source file, with no project, no linking multiple files,

and nothing to set up first.



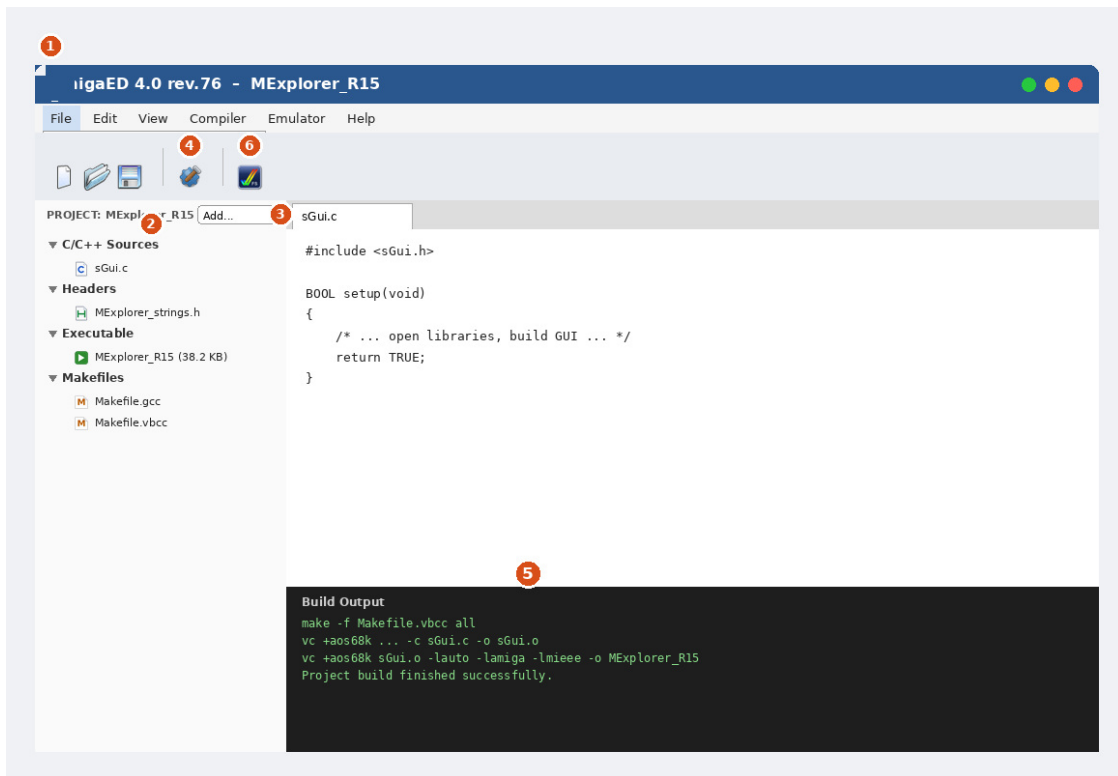
The editor in Ad-hoc mode: numbers match the steps below.

1. File → New (or Open...) to create or open a single .c/.cpp file. No project is involved.
2. Write or edit your code in the editor tab, with full syntax highlighting.
3. Pick your target toolchain and OS from the compiler dropdown in the toolbar (e.g. vbcc / OS 3.x, or m68k-amigaos-gcc / OS 1.3).
4. Click the Compile toolbar icon (or Compiler menu) to build just this file.
5. Check the Compiler Output panel at the bottom, which shows the exact compiler/linker commands used and whether the build succeeded.
6. Optionally, click Start Emulator to launch UAE and try your freshly built program right away.

Note: Ad-hoc compilation always builds and links just the one open file. If your program needs more than one source file, switch to Project Driven Compilation instead (see next chapter).

Mode 2: Project Driven Compilation

Use this for anything with more than one source file, a GUI (ReAction or MUI), or a program you will want to rebuild and maintain over time.



The Project panel and toolbar: numbers match the steps below.

1. File → New Project... to start from a template (Empty C, AmigaShell, AmigaOS 1.3, AmigaOS 3.x, ReAction, or MUI), or File → Import existing Project... to bring in a folder of sources AmigaED does not know yet.
2. The Project panel on the left shows all your files, automatically sorted into categories: C/C++ Sources, Headers, Assembler Sources, AmigaGuide, Installer Scripts, Executables, Other Files, and the auto-generated Makefiles.
3. Add further files any time with Add files to Project..., or by dragging them straight onto the Project panel.
4. Click Build Project. AmigaED (re)generates the Makefiles for every configured toolchain and runs the build for you.
5. Watch the Build Output panel. On success, the compiled program appears in the Project panel Executable category, together with its file size.
6. Click Start Emulator to launch UAE and test your program.

Note: A project's Makefiles are regenerated automatically every time you add or remove a file, and also right before every Build Project - so a plain content change (e.g. adding floating-point usage to an already-tracked file) is always picked up too, not just file-list changes. Both vbcc and gcc/g++ Makefiles are generated side by side, plus a SAS/C Makefile for building later on a real Amiga.

Note: Build Project also saves any open project file with unsaved changes first - including a hand-edited Makefile, not just source files - since both the Makefile regeneration above and the compiler itself always read straight from disk. Whether this happens automatically or asks first is controlled by Prefs → Project → "Save Project Files Automatically" (see below).

Closing a project

File → Close Project closes every tab belonging to the current project (prompting to save any with unsaved changes first, exactly like switching to a different project), then returns to the “no project loaded” state. Cancelling a save prompt leaves the project and all its tabs open untouched.

Automatic program icons

If Prefs → Misc → “create icon” is checked, AmigaED writes its own built-in, multi-coloured Amiga tool icon next to your compiled program - both for a project build and for an ad-hoc single-file compile. No default icon file needs to be configured anywhere. The icon's stack size is set automatically for the target: 4096 bytes for AmigaOS 1.3, 8192 for AmigaOS 3.x, 16000 for a ReAction project, and 34000 for a MUI project.

Cleaning a project

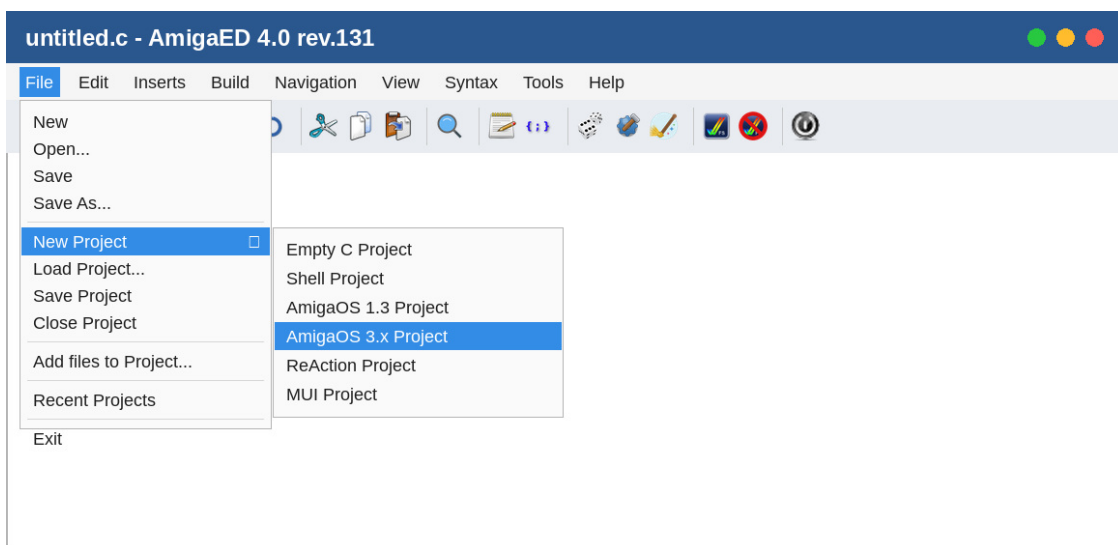
Build → Clean Project removes the current project's build artifacts - every object file, the compiled executable, and its icon - by deleting each one directly and individually, listing every file it removes in the Compiler Output panel. It does not invoke “make clean” through a shell.

Quick Start: From Template to Clean Build

A concrete, illustrated walk-through of Mode 2 above, start to finish: creating a new project from a template, building it, and cleaning it up again.

1. Create the project

File → New Project offers six templates: Empty C Project, Shell Project, AmigaOS 1.3 Project, AmigaOS 3.x Project, ReAction Project, and MUI Project. The ReAction and MUI templates already build a small working window with a File menu (About/Quit) - a genuine starting point, not just an empty shell.



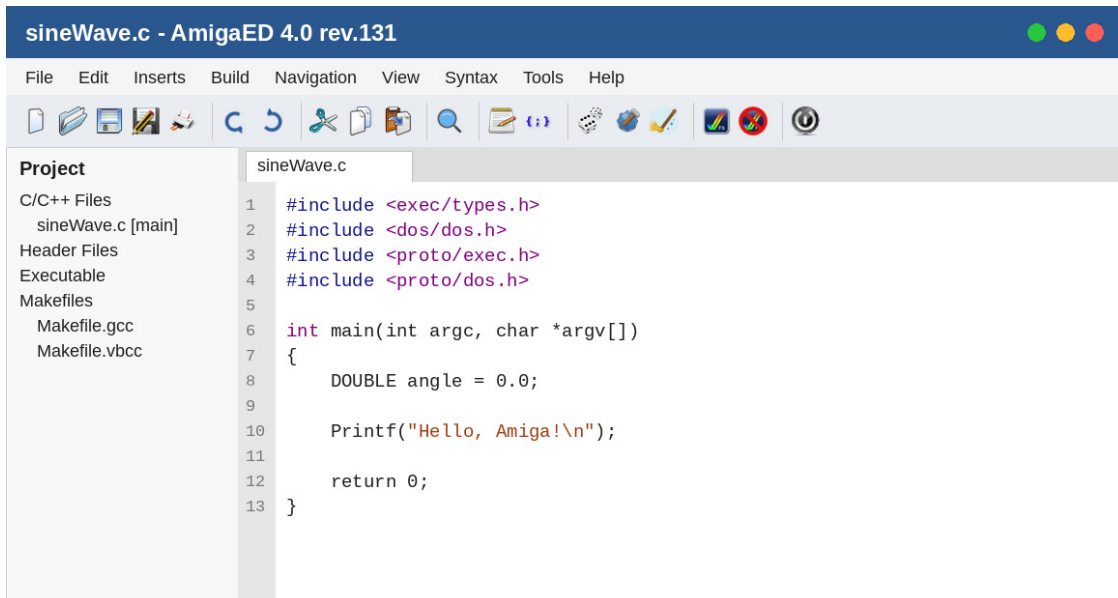
Picking “AmigaOS 3.x Project” from the New Project submenu.

After picking a template, AmigaED asks for a target directory and a project name, then (for AmigaOS 1.3/3.x, ReAction, and MUI templates) confirms the default compiler/linker switches for that target before

creating anything - the same switches listed in the table further below, pre-filled and ready to adjust.

2. Write your code

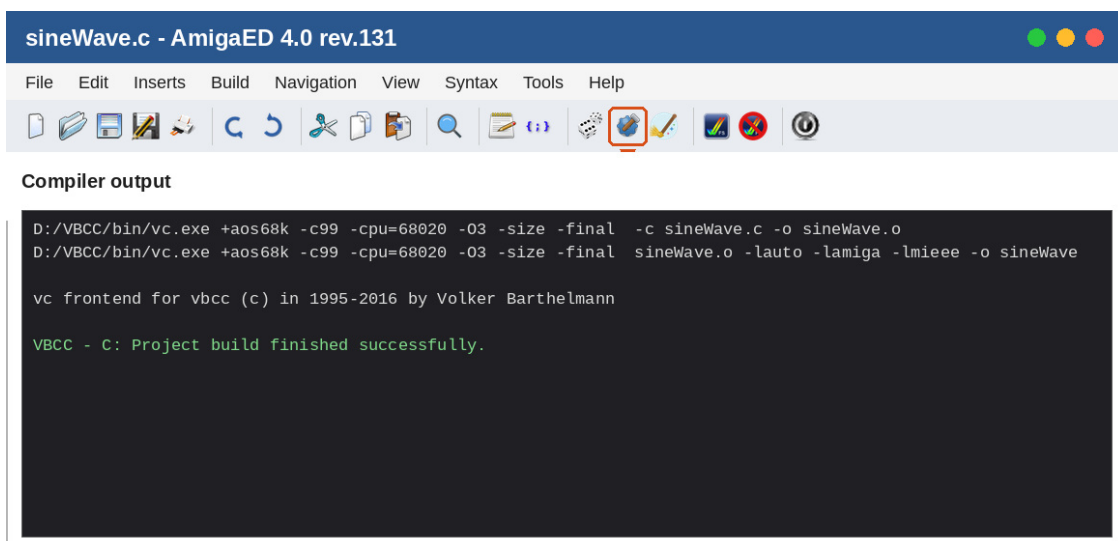
The generated main file opens automatically, already including the right Amiga NDK headers for the chosen template. The Project panel on the left shows the file, the auto-generated Makefiles, and the (still empty) Executable category.



A freshly created AmigaOS 3.x project, ready to edit.

3. Build it

Click the Build Project toolbar icon (or Build → Build Project). AmigaED regenerates the Makefiles fresh, then runs the currently selected compiler (VBCC or GCC/G++, picked in the status bar) against them. If your code uses `float/double` - or the Amiga-typical uppercase `FLOAT/DOUBLE` typedefs, both are recognized - the right math library is linked in automatically; nothing to configure by hand.



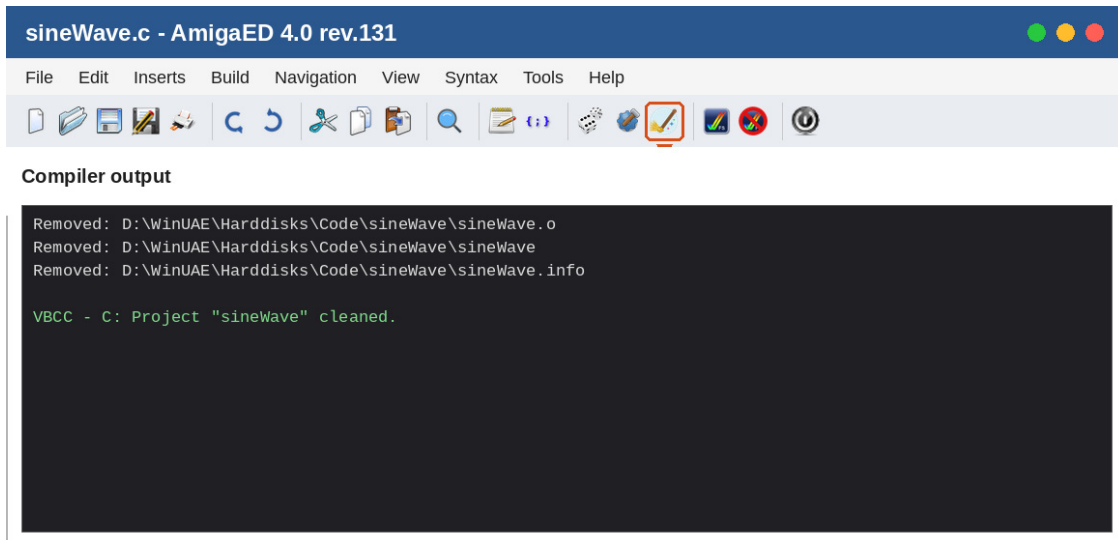
A successful build, reported in the Compiler Output panel.

See “The Compiler Output Pane” below for how to jump straight from an error or warning line to the

offending source line.

4. Clean up

Click the Clean Project toolbar icon (or Build → Clean Project) to remove the build artifacts again - every object file, the compiled executable, and its icon - each one deleted directly and listed individually in the Compiler Output panel.

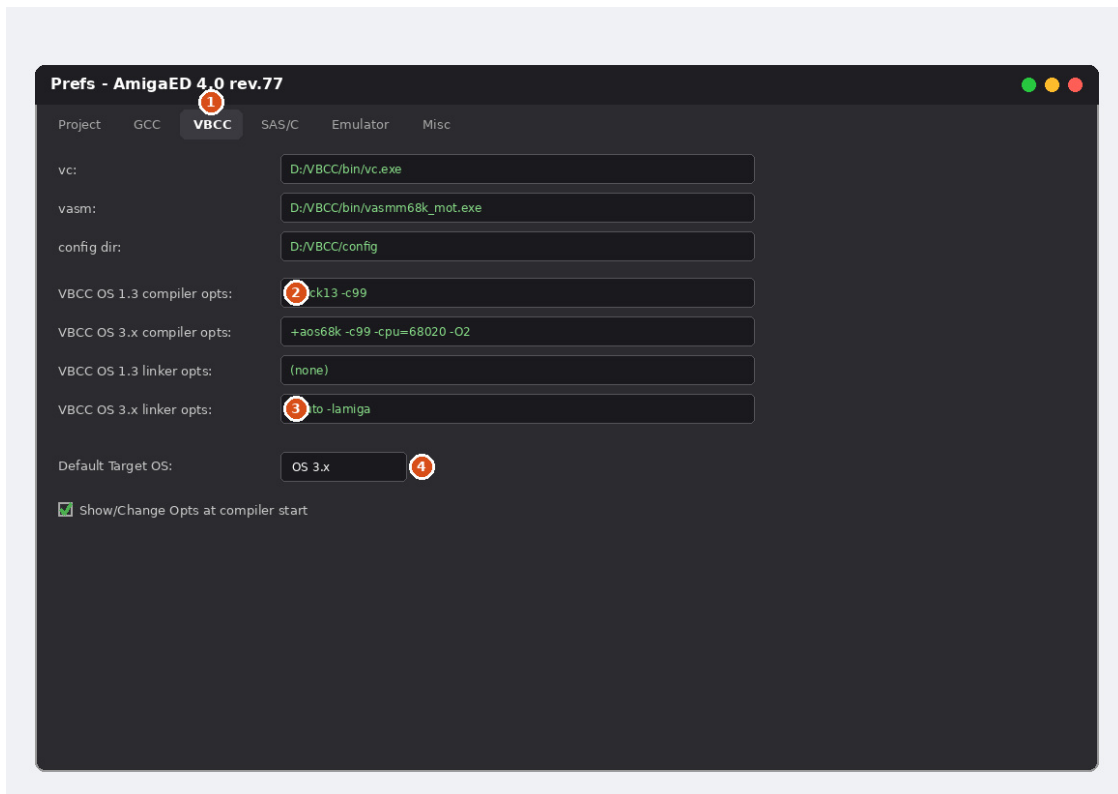


Clean Project's output: every removed file is listed by name.

Note: Your source files, the .aep project file, and the Makefiles themselves are never touched by Clean Project - only the files a build actually produces.

The Preferences Editor

File → Prefs... opens AmigaED settings, organized into tabs: Project (author/email/website, default projects folder), GCC, VBCC, and SAS/C (paths to each toolchain and their compiler/linker options, with separate fields for AmigaOS 1.3 and AmigaOS 3.x targets), Emulator (path to your UAE executable, and separate OS 1.3/3.x configs), and Misc (application style/theme, whether to create a program icon, and other editor behaviour).



The VBCC tab, as an example: GCC and SAS/C follow the same pattern.

1. Tabs across the top switch between Project, GCC, VBCC, SAS/C, Emulator, and Misc settings.
2. Compiler options are kept separate for AmigaOS 1.3 and AmigaOS 3.x, since the two targets typically need different flags (see the tables below).
3. Linker options are likewise separate per target OS. This is where a math library gets added automatically when your project uses floating point (see note below).
4. Default Target OS sets which of the two option sets above is used when you do not explicitly pick one.

Automatic floating-point handling: if AmigaED detects float or double - or the Amiga-typical uppercase FLOAT/DOUBLE typedefs, both are recognized - anywhere in your project's own C/C++ sources, it automatically adds the right math library to the generated Makefiles for you: -lm for gcc/g++, -lmieee for vbcc, and MATH=IEEE for SAS/C (only if no MATH= mode is already set). You do not need to add these yourself.

Saving before a build

Prefs → Project → “Save Project Files Automatically” (directly below “Projects root”) controls what Build Project does when an open project file - a source file or a hand-edited Makefile alike - has unsaved changes: checked, it saves everything immediately with no prompt; unchecked (the default), it asks once, listing every affected file, with the option to save them, build anyway from whatever is already on disk, or cancel the build entirely.

Themes

Prefs → Misc → “Default application style” (or the View → Theme menu, which changes it immediately without opening Prefs at all) lists every native style available on your platform, plus three synthetic ones:

Dark, Workbench 1.3 (the classic blue Kickstart 1.3 desktop look, sampled from a real screenshot), and **Workbench 3.1** (the plainer, neutral grey AmigaOS 3.x look, also sampled from a real screenshot). View → Theme shows all of them as a single, mutually exclusive list - picking one applies it immediately, everywhere: the application chrome, every open editor tab, and the Compiler Output pane's error/warning colours all update together.

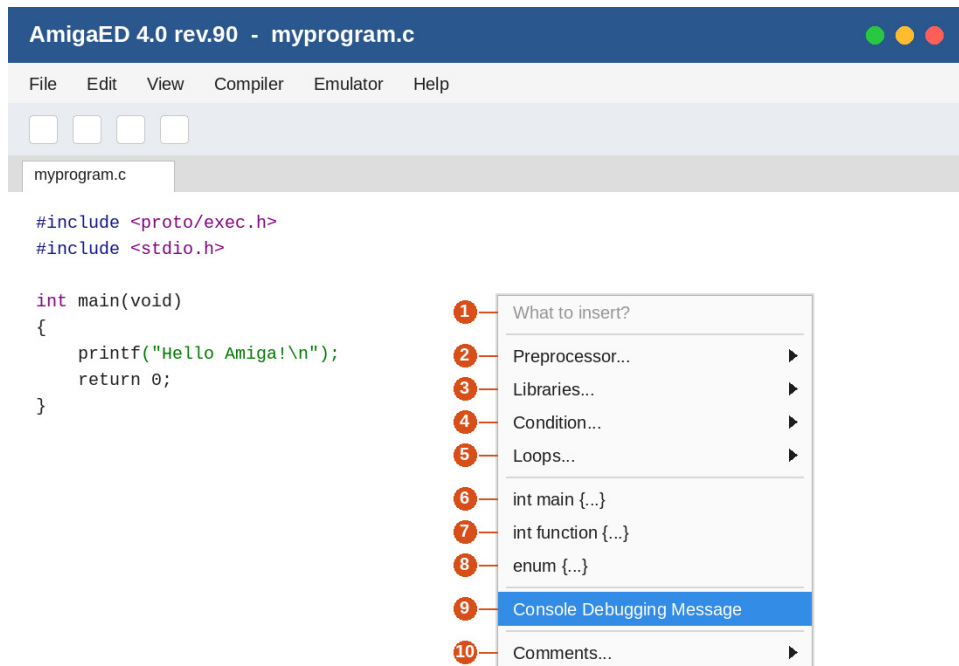
The emulator and running instances

If you close AmigaED while its emulator is still running, it asks whether to leave the emulator open or shut it down too - handy since starting an emulator takes real time you can skip on the next session. Starting the emulator also checks, via the operating system's own process list, whether a matching emulator process is already running - whether AmigaED itself started it (e.g. one left open like this) or not - and asks before launching a second instance rather than doing so unconditionally. Start/Stop Emulator (toolbar and Tools menu) automatically reflect whichever of these is currently true.

Known issue: FS-UAE under WSL2/WSLg: if you run the Linux build of FS-UAE inside WSL2 (Windows Subsystem for Linux), its mouse capture is unreliable - both the host and the emulated Amiga cursor can remain visible at once, drifting apart as you move the mouse, no matter how FS-UAE's own configuration options for mouse grabbing are set. This traces back to a known WSLg limitation with relative mouse input (see microsoft/wslg issue #240), not a bug in FS-UAE, AmigaED, or your configuration - confirmed by the same FS-UAE configuration working correctly on a real Linux machine. If you need FS-UAE specifically inside WSL2, running an X server on the Windows side (e.g. VcXsrv) instead of relying on WSLg's own GUI integration is the more promising workaround; otherwise, running FS-UAE on a native Linux machine (or natively on Windows, using the Windows build of FS-UAE) avoids the issue entirely.

The Editor's Context Menu

Right-click anywhere inside the code editor to open a context menu of quick code-insertion shortcuts. It mirrors the main Inserts menu (menu bar) exactly: anything listed here is also available there, under the same name.



The editor context menu, opened over a source file: numbers match the entries explained below.

- 1. What to insert?** – a disabled heading line, not a clickable entry. It is only there to label the menu.
- 2. Preprocessor...** – a submenu of preprocessor-related inserts: `#include`, `#define`, `#ifdef`, `#ifndef`, `#if defined(...)`, **Amiga #include files** (a ready-made block of the most common Amiga NDK headers), **Identify Amiga compiler** (a chain of `#if defined(...)` checks generating a `compiler_string` constant naming whichever compiler is in use), and **Amiga C version string** (a `$VER:` tag).
- 3. Libraries...** – `OpenLibrary()` and `CloseLibrary()`, each inserting a ready-to-edit `dummy.library` Open/Close template with error handling.
- 4. Condition...** – `if(..) {...}` and `if(..) {...} else {...}` skeletons.
- 5. Loops...** – `while(...){...}`, `for(...){...}`, `do...{...}while(...)`, and `switch(...)` (already including a case `dummy: break;`).
- 6. int main {...}** – inserts a complete `main()` function skeleton; disabled once the file already contains one.
- 7. int function {...}** – inserts a function prototype plus a matching function skeleton, ready to cut and paste into place.
- 8. enum {...}** – inserts a complete C enumeration skeleton (`SomeEnum` with three example values).
- 9. Console Debugging Message** – inserts an `if (myDebug) {...}` debugging block; the caret lands right inside it, ready to type. Needs `#define myDebug TRUE` earlier in the file, already included near the top of every "New Project" template.
- 10. Comments...** – **Fileheader comment...**, C-style single/multi line comments, a C++ style single line comment, and a C-style line-dividing comment.

Note: Most entries insert their skeleton right at the caret position and leave the caret ready to keep typing, either on a new line after the inserted block, or (for Console Debugging Message) right inside it. This context menu is

built from the exact same actions as the main Inserts menu, so the two are always in sync.

The Compiler Output Pane

Every compile - ad-hoc (Mode 1) or project (Mode 2), VBCC or GCC/G++ - prints its output into the Compiler Output pane at the bottom, with two things making it easier to work through: colour-coded diagnostics, and click-to-jump.

Colour-coded errors and warnings

Every recognized error line is highlighted in red, every warning in yellow/orange - bold text on a tinted background, using a colour scheme matched to the current theme (see Themes above) so it stays readable whichever one is active. Purely informational lines (compiler invocation echoes, "In file included from...", GCC's own source-snippet/caret lines) are deliberately left uncoloured.

Click to jump

Click any error or warning line to jump straight to the offending file and line: the file opens in a tab (or is simply activated, if already open) and the caret moves to the exact reported line and column.

Compiler	Recognized format
VBCC	error 9 in line 24 of "file.c": message
GCC/G++	file.c:24:5: error: message

Note: An error reported inside a generated intermediate file (e.g. VBCC reporting an assembler-level problem in the .asm file it produced from your .c source) still jumps correctly - to that intermediate file, exactly as the compiler itself reported it.

Suggested Compiler & Linker Switches

These are AmigaED own built-in defaults: a solid, well-tested starting point for each toolchain and target. You are always free to add your own project-specific flags in the same Prefs fields alongside them.

m68k-amigaos-gcc (GCC/G++)

OS 1.3 compiler	-Wall -O2 -mcrt=nix13	Common warnings, optimize, link against libnix built for Kickstart 1.3.
OS 3.x compiler	-Wall -O2 -noixemul	Common warnings, optimize, use the AmigaOS-native C runtime instead of ixemul.library.
OS 1.3 linker	(none)	The OS 1.3 template is console-only and needs no extra libraries by default.

OS 3.x linker	-lamiga	Needed for Workbench-capable programs (ReAction/MUI), added automatically by AmigaED OS 3.x/ReAction/MUI templates.
---------------	---------	---

vbcc (vc)

OS 1.3 compiler	+kick13 -c99	Named vbcc config for Kickstart 1.3, C99 language mode.
OS 3.x compiler	+aos68k -c99	Named vbcc config for AmigaOS 2.0+/3.x, C99 language mode.
OS 1.3 linker	(none)	Console-only OS 1.3 template needs no extra libraries by default.
OS 3.x linker	-lauto -lamiga	vbcc own amiga.lib equivalent, needed for Workbench-capable (ReAction/MUI) programs.

Optional additions many users like to add for a release build once things compile cleanly: -cpu=68020 (or higher), -O2/-O3 (optimize), -size (optimize for code size), -final (disable runtime checks). These are not part of AmigaED own defaults since they trade off compatibility or debuggability for speed/size.

SAS/C

SAS/C only runs on a real Amiga or emulator, so AmigaED never invokes it itself: it just generates a Makefile.sc alongside the others for later, manual use there. The SCOPTS field in Prefs starts empty; AmigaED still adds MATH=IEEE automatically if your project needs it.

Recommendations for Amiga C Programmers

A short, curated list of literature and tools the Amiga C/ReAction/MUI development community particularly recommends. None of this is bundled with AmigaED, but all of it pairs well with it.

Literature

Amiga ROM Kernel Reference Manual: Exec / Libraries and Devices / Includes and Autodocs

The original Commodore-Amiga reference series, still the authoritative source for Exec, Intuition, and the rest of the classic system libraries. Out of print; free scanned copies are archived at archive.org.

Amiga ROM Kernel Reference Manual: AmigaDOS – Thomas Richter

A modern, RKR-style reference specifically for AmigaDOS/dos.library. Free download from Aminet; a printed edition has also been available via lookbehindyou.de.

ROM Kernel Reference Manual: Changes & Additions – Camilla Boemann & Jason Stead

An ongoing (WIP) volume covering everything added to the Amiga OS from Release 2 through 3.2. Free PDF from developer.amigaos3.net.

Erik Bartmanns Amiga Programmierbuch

C programming, cross-compiling with vbcc and ReAction, on an Amiga 4000 (or Amiga Forever). Book: 32 EUR,

e-book: 19.99 EUR, via bombini-verlag.de.

Erik Bartmanns Amiga 1200 Buch

Machine language, C programming, and Shell: 717 pages across 28 chapters, focused on the Amiga 1200.

Book: 64 EUR, e-book: 29.99 EUR, via bombini-verlag.de.

Tools

Rebuild – Darren Coles

A ReAction GUI builder (a from-scratch remake of the classic ClassMate), generating C or Amiga E code. Public domain. Source: github.com/dmcoles/ReBuild

Codecraft – Camilla Boemann

Codecraft is a powerful IDE for developing software natively on the Amiga. Codecraft makes it easy for you as a developer to write code, then build, execute and debug your resulting program. And everything is at your fingertips in one unified user interface. A native IDE for AmigaOS 3.2.3 and later. Source: gitlab.com/boemann/codecraft, <http://boemann.dk/codecraft/>

MUIBuilder

A user interface designer for the MUI and Zune toolkits. GPLv3/LGPLv3. Source: sourceforge.net/projects/muibuilder

Prices, links, and availability change over time. If something above is no longer reachable, a web search for the title and author is usually the fastest way to find its current home.